Fundamentals of Java Programming - Part 1

Java Application Structure

Identifiers

Datatypes

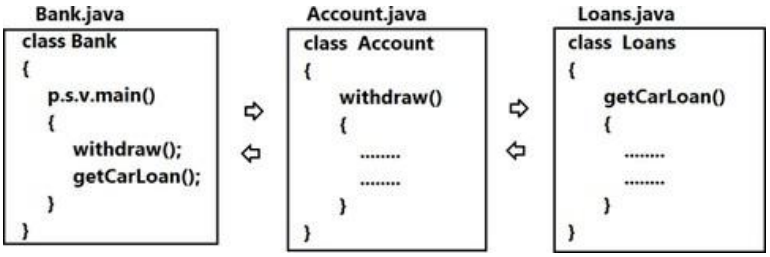
Type Casting

---

i Java Application Structure

- · Java application is a collection of Source files.

- Source file consists one or more classes.
- Class contains Variables and Methods.
- Java application runs by JVM.
- JVM invoke main() method to start application execution.
- Application starts with one class in which main() method has defined.



How to Compile and Run Java Program from Command Prompt?

1. Open Notepad, Write Code and Save file with .java extension.
2. Open Command prompt(CUI based OS).
3. Go to file location.
4. Compile using command – javac
5. Run using - java

```
public class  FirstProgram {
        public static void main(String[] args) {

          System.out.println("Hello World!");
        }
  }
```

```
C:\Users\SHIVA\Desktop\java>javac FirstProgram.java

C:\Users\SHIVA\Desktop\java>java FirstProgram
Hello World!

C:\Users\SHIVA\Desktop\java>
```

ii Identifiers and Keywords

Keywords :

These are pre-defined words which is having fixed meaning and purpose, we cannot change it.

We have 52 keyword in java , **ex : public, class, private, int, float, short, byte, long, double and boolean**..... **so on**

A **Java identifier** is a name assigned to a package, class, interface, method, or variable, allowing programmers to reference and interact with these elements throughout the code. Meaningful and descriptive identifiers improve code readability and maintainability. To ensure clarity, it's essential to adhere to standard Java naming conventions, which help make the code more understandable and professional across teams and projects.

Rules to create identifier:

1. Identifier should not start with digit and Special Character excpet underscore(_) or a dollar sign ($).
2. Spaces not allowed in identifier.
3. We cannot use the Java reserved keywords as an identifier.

iii Datatypes

Datatype

- It is used to create a memory block to store the data in application.
- Data type describes,

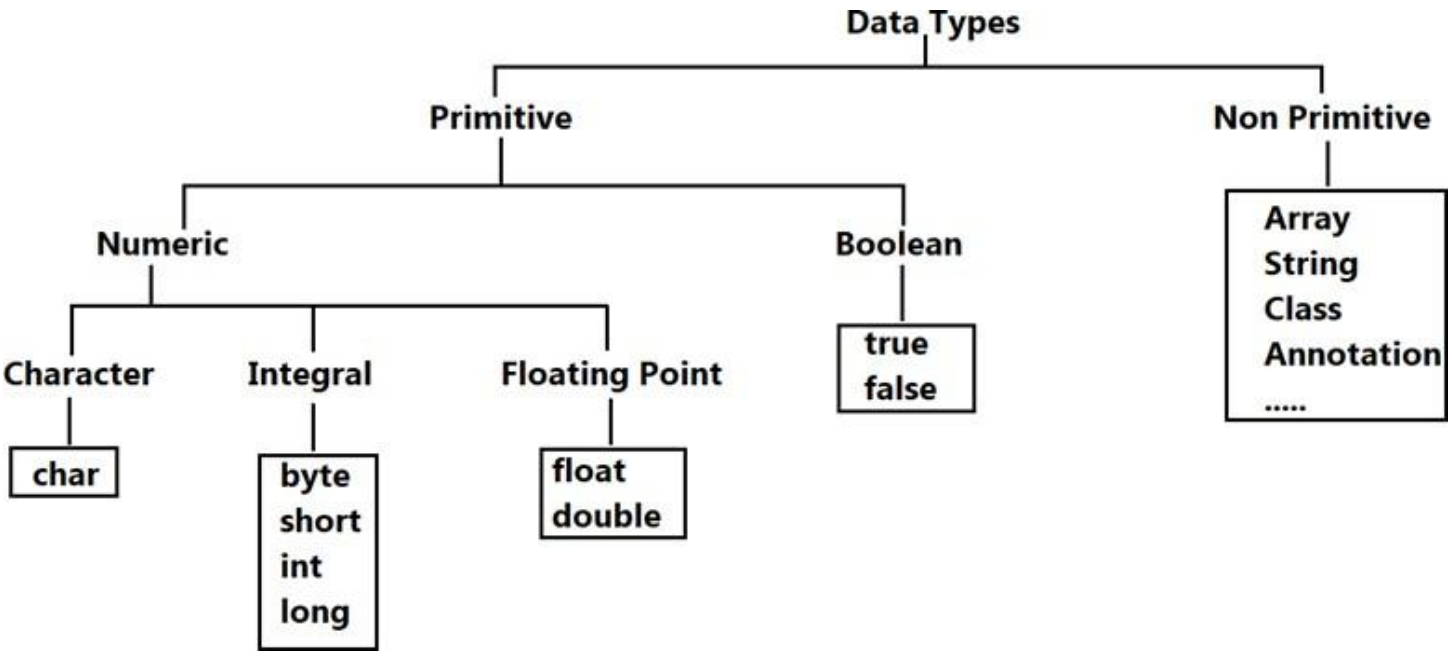
1. The size of memory allocated to variable.
2. The type of data allowed to store into variable.

Syntax: data_type variable_name = value ;

Example: int sum = 0;

float balance = 3000.0; String name = “Amar”;

Data types classified into Primitive and Non-Primitive types.



Non-Primitive Datatype:

- It is known as User-defined datatype.
- We have some pre-defined non-primitive datatypes such as String, StringBuffer, StringBuilder, Integer, Boolean etc.

- **Size :** The size is undefined for Non-primitive data types.

Ex: String, StringBuffer, Car, Bus all are Non-primitive datatypes.

String : It is a collection of characters and doesn't have specific size.

we can use this way ====> String str = "Hello this is java fundamentals";

So, String is non-primitive but majorly used as simple data.

Limits of Primitive Datatype:

- A data type limit describes the minimum and maximum values we can store into a variable.
- Every primitive data type is having size and it is represented in bytes
- One byte equals to 8 bits.

Integer types: We have 4 integer types among eight available primitive types in java.

Note:

- For every primitive type, there is a corresponding wrapper class.
- Wrapper classes providing pre-defined variables to find Size and Limits.

Program to print sizes of Integer types:

```
class IntegersDatatypeSizes {

        public static void main(String[] args) {

                System.out.println("Byte size : " + Byte.SIZE/8 + " bytes");
                System.out.println("Short size : " + Short.SIZE/8 + " bytes");
                System.out.println("Integer size : " + Integer.SIZE/8 + " bytes");
                System.out.println("Long size : " + Long.SIZE/8 + " bytes");

        }

}
```

Program to print Limits of Integer types:

```
class PrimitiveLimits {

        public static void main(String[] args) {

                System.out.println("Byte min val : " + Byte.MIN_VALUE);
                System.out.println("Byte max val : " + Byte.MAX_VALUE);
                System.out.println("Short min val : " + Short.MIN_VALUE);
                System.out.println("Short max val : " + Short.MAX_VALUE);

        }

}
```

Character data type: Character type variable is used to store alphabets, digits and symbols.

```
class Chars{

                public static void main(String[] args) {

                char x = 'A';

                char y = '5'; char z = '$';

                System.out.println("x val : " + x + "  y val : " + y + "  z val : " + z);

        }

}
```

Note: We must represent Character with Single Quotes in Java

ASCII : Representation of each character using constant integer value.

A-65 B-66	a-97 b-98	0-48	*-34 #-35
...	....	1-49	$-36
...	....	..	...
... Z-90	....	..	...
	z-122	.. 9-57	...

iv Type Casting

Based on Datatypes type casting is Classified into two types :

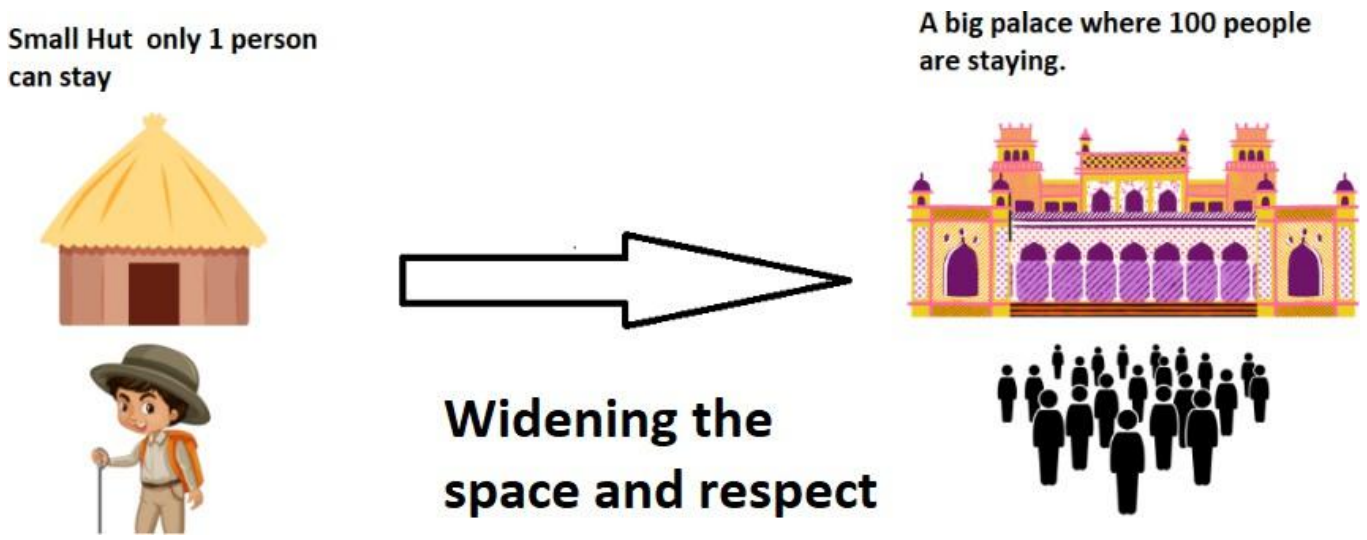
1. Primitive Type Casting
2. Non - Primitive Type Casting (After Inheritance will be continued).

Primitve Type Casting :

It is again classified to Two types :

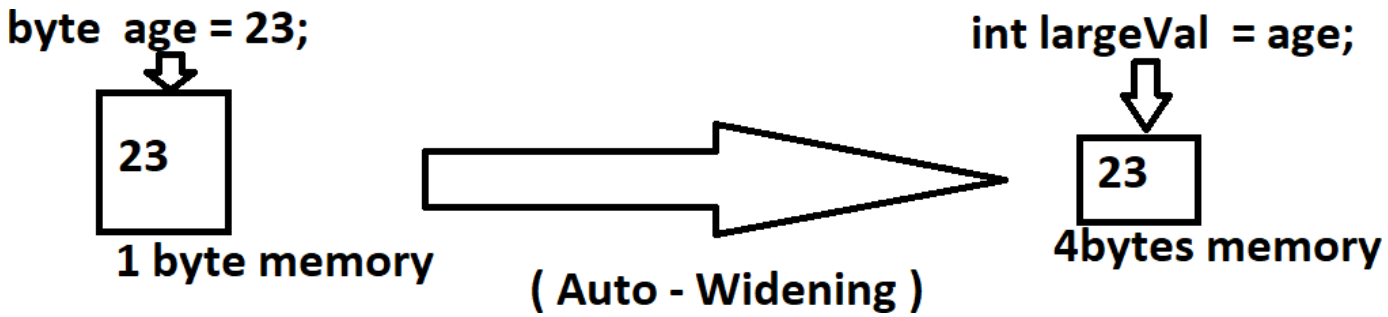
1. **Auto-Widening :** Converting lesser datatype value to higher datatype value is called Auto-Widening.
2. **Explicit Narrowing** : Converting higher datatype value to lesser datatype value is called Explicit Narrowing.

Auto-Widening :



In the above example sending one person to Big palace can accept and the space is increased for person to stay.

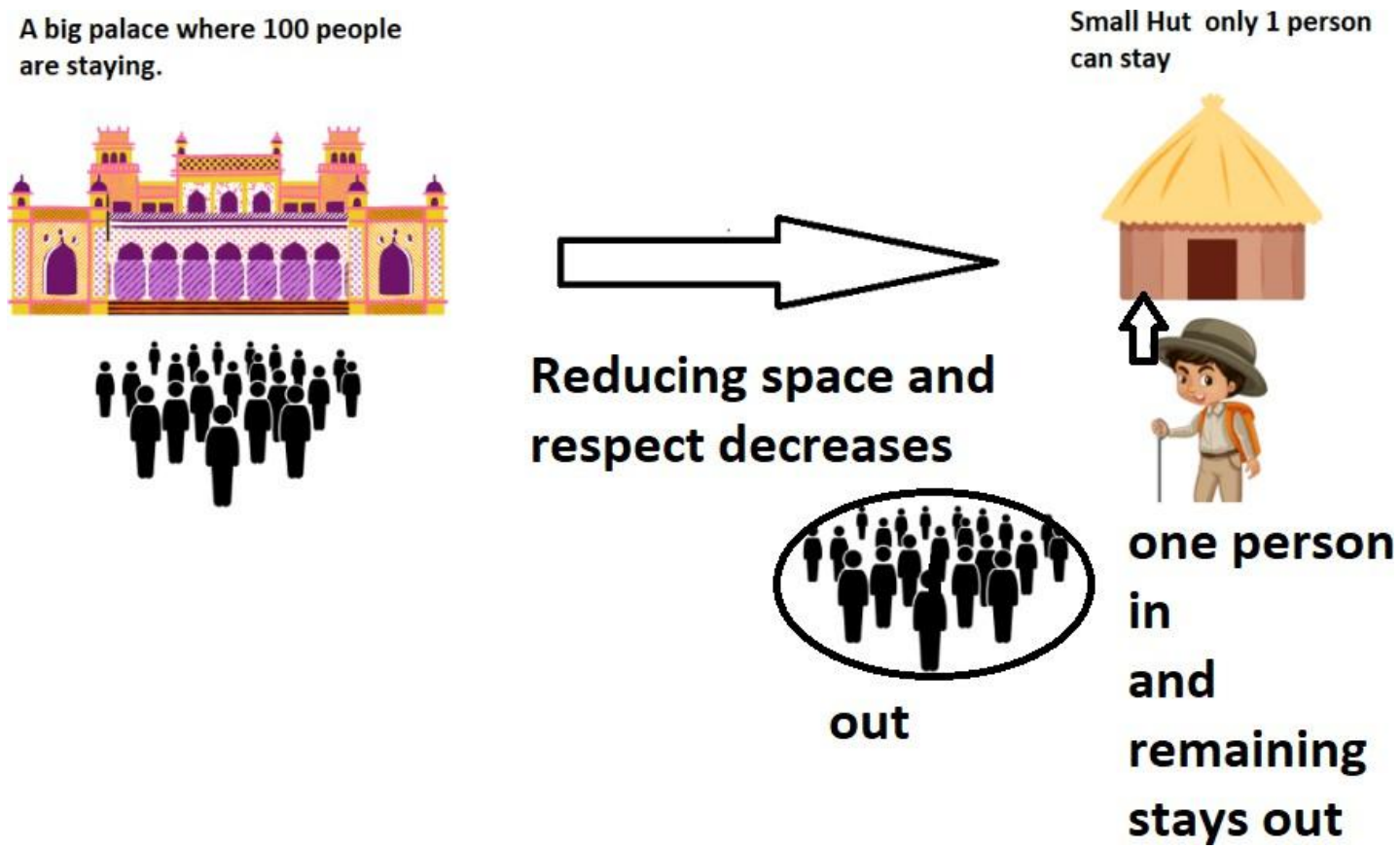
As the above example can be used as best analogy for auto-widening.



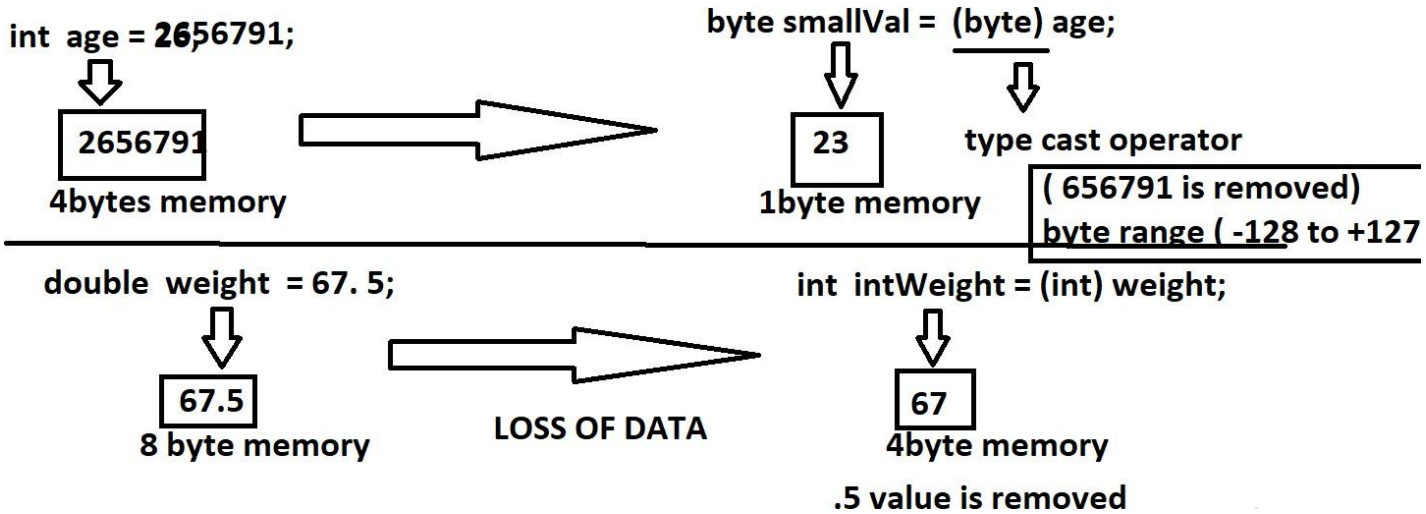
Compiler written code => // int largeVal = (int) age;

Note : Compiler adds the type cast operator automatically so, it is called **<u>auto-widening</u>.**

Explicit Narrowing :



The above example is best analogy of Explicit Narrawing.

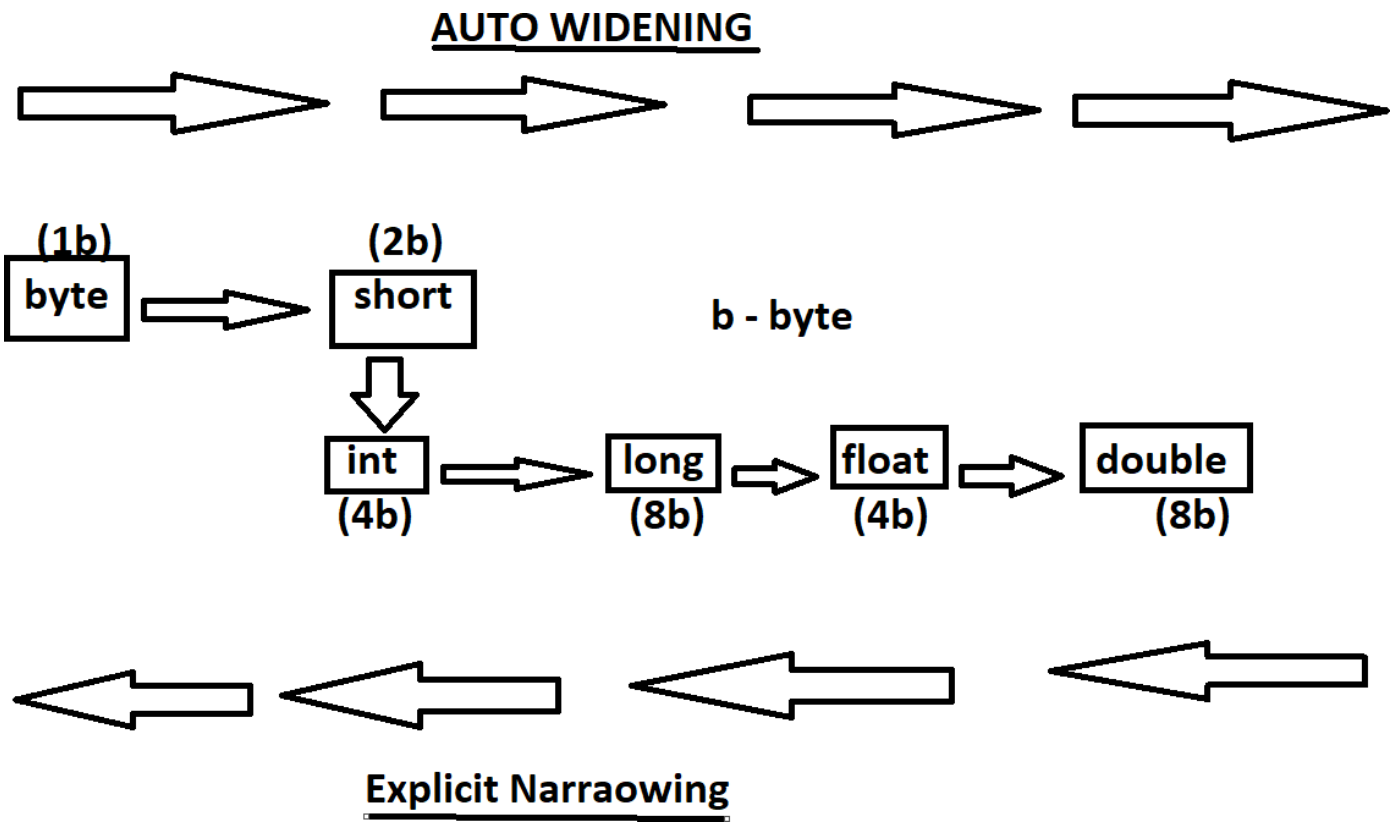


In explicit narrowing we are adding **<u>type cast operator explicitly.</u>**

```
int age = 2656791;

byte smallVal = (byte) age;

System.out.println(smallVal); // 23  LOSS OF DATA
```



Note: Boolean cannont be using in type casting it has only one bit memory and used only for representation. (True or False)

Exceptional Cases :

char datatype accepts integer because internally every character will be converted to it's **ASCII value** so it accepts integer and prints the character of that ASCII value.

Convert Numbers to Character: this conversion must be done manually

```
char ch = 97 ;

System.out.println("char : "+ch); // char : a

ch = (int) 97.53f;

System.out.println("char :  "+ch): // char : a

ch = (int) 97.5 ;

System.out.println("char : "+ch); // char : a
```

Convert Character to Integer: this conversion happens implicitly when we assign character to integer type.

```
int ch = 'A';

System.out.println("Value : "+ch); // Value : 65
```

Convert Upper case to Lower case Character:

Upper Case		Lower Case
A = 65	+32 →	a = 97
B = 66		b = 98
..	← -32	..
..		..
Z = 90		z = 122

```
char up = 'A';

char lw = (char)(up+32);

System.out.println("Result : " + lw); // Result : a
```

Convert Digit to Number: We can convert character type digit into number as follows

Digits ASCII	Number
'0' = 48	'0' - 48 --> 48-48 = 0
'1' = 49	'1' - 48 --> 49-48 = 1
..	..
..	..
'9' = 57	'9' - 48 --> 57-48 = 9

Decimal Numbers:

- We store decimal values using 2 types float and double.

- We must specify the float type with precision f or F
- By default, double precision d or D will be added at end.

Formatting Output

Introduction: It is important to format the output before display the results to end user. Proper formatting makes the user more understandable about program results.

We always display results in String format. To format the output, we concatenate the values such as int, char, double, boolean with messages (string type) as follows:

Syntax	Example
int + int = int	30 + 20 = 50
String + String = String	“Hello” + “James” = “HelloJames” “30” + “60” = 3060
String + int = String	“James” + 007 = “James007” “987” + 428 = “987428”
String + int + int = String	“Product = “ + 30 + 20 = ”Product = 3020”
String + (int + int) = String	“Value = “ + (30 + 20) = Value = 50
String + double = String	“final = “ + 2.3f = ”final = 2.3f”
String – int = Error	

ex: 1

```
int a=10, b=20;
System.out.println(a + “ , “ + b);
```

Output: 10, 20

ex: 2

```
int a=10, b=20;
System.out.println("a = " + a + “ , “ + "b = " + b);
```

Output: a = 10, b = 20

ex: 3

```
int a=10, b=20;
int c=a+b;
System.out.println("Sum of " + a + " and " + b + " is " + c);
```

Output: Sum of 10 and 20 is 30

ex: 4

```
int a=10, b=20;
int c=a+b;
System.out.println(a + " + " + b + " = " + c);
```

Output: 10+20=30

Reading Input Using Scanner Class

Scanner Class:

java library provides Scanner class, we can take input like integers, characters, strings, double values from the user.

- Different methods to read different values such as nextInt(), next(), nextDouble()…
- We need to specify the Keyboard(System.in) while creating Scanner class object.

```
Scanner scanner = new Scanner(System.in);
```

- We access all the methods using object reference variable of Scanner.

Reading integer value: nextInt() method read and returns an integer value.

```
import java.util.Scanner;
class ReadInt
{
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter integer : ");
        int n = scan.nextInt();
```

```
        System.out.println("n value is : " + n);
    }
}
```

Output:

Enter integer : 10

n value is : 10

Reading Boolean value: nextBoolean() method returns Boolean value that we entered.

```
import java.util.Scanner;
class ReadBoolean
{
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter boolean value : ");
        boolean b = scan.nextBoolean();
        System.out.println("b value is : " + b);
    }
}
```

Output: Enter Boolean value : true

b value is : true

Reading String: using next() method we can read single word string from the user.

```
import java.util.Scanner;
class Code
{
```

```
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter your name : ");
        String name = scan.next();
        System.out.println("Hello : " + name);
    }
}
```

Output: Enter your name: Amar

Hello : Amar

Reading character: There is no method in Scanner class to read single character, hence we read first character of String to read single character as follows

```
import java.util.Scanner;
class Code {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter character : ");
        char ch = scan.next().charAt(0);
        System.out.println("character is : " + ch);
    }
}
```

Output: Enter character : Z

character is : Z